

A.L.I.C.E.

Assistente Intelligente per la Cura degli Elderly

Documentazione di Progetto Piattaforma Web per il Monitoraggio Remoto di Anziani

Anno Accademico:
2025/2026

Esame:
PROGRAMMAZIONE PER IL WEB

Realizzato da

Manuel	Manna	803080	m.manna6@studenti.uniba.it
Alessio	Mininni	798191	a.mininni14@studenti.uniba.it
Andrius	Tamuliunaite	800256	a.tamuliunaite@studenti.uniba.it

Indice

1	Descrizione Informale del Contesto e degli Obiettivi	3
2	Individuazione degli Stakeholder e dei Goal	4
2.1	Stakeholder e Goal	4
2.1.1	Goal Operatore Sanitario	4
2.1.2	Goal Paziente Anziano	4
2.2	Diagramma di Contesto	5
2.3	Scambio di Valore	6
3	Stato dell'Arte, Analisi della Concorrenza e Analisi "Make or Buy"	7
3.1	Stato dell'Arte	7
3.2	Analisi della Concorrenza	7
3.2.1	MyTherapy	7
3.2.2	ADiLife	7
3.3	Analisi "Make or Buy"	8
4	Modello Informativo	9
4.1	Diagramma EER	9
4.2	Modello Relazionale	11
4.3	Stima Dimensionale del Database	13
4.3.1	Stima per entità	13
4.3.2	Tabella di popolamento e crescita	13
4.3.3	Modello di visibilità	14
5	Modello di Navigazione	16
5.1	Albero di Navigazione	16
5.2	Legenda Tipi di Vista	17
5.3	Descrizione delle Viste	17
5.3.1	Area comune	17
5.3.2	Area Operatore Sanitario	17
5.3.3	Area Paziente Anziano	17
5.4	Principi di Navigazione	18
6	Modello Architetture	19
6.1	Architettura Hardware	19
6.2	Architettura Software	20
6.2.1	Stack Tecnologico	20
6.2.2	DBMS	20
6.2.3	Browser e Sistemi Operativi Supportati	21
7	Aspetti Implementativi	22
7.1	Struttura del Progetto	22
7.1.1	Pagine (app/)	22
7.1.2	Componenti Riutilizzabili (components/)	23
7.1.3	Lato Server (lib/ e actions)	23
7.2	Sicurezza	24
7.2.1	Autenticazione	24

7.2.2	Row Level Security (RLS)	24
8	Test	25
9	Conclusioni	26
9.1	Sviluppi futuri	26
10	Matrice RACI	27
10.1	Matrice – Documentazione	27
10.2	Matrice – Sviluppo	28

1 Descrizione Informale del Contesto e degli Obiettivi

La nostra applicazione mira ad offrire una piattaforma web innovativa per il monitoraggio remoto e l'assistenza domiciliare di pazienti anziani.

L'invecchiamento della popolazione è una realtà che richiede soluzioni tecnologiche accessibili. Molte delle piattaforme esistenti risultano troppo complesse per l'utenza anziana, oppure troppo semplificate per essere utili ai professionisti sanitari. Il nostro obiettivo è colmare questo divario. Puntiamo a realizzare **A.L.I.C.E.** (Assistente Intelligente per la Cura degli Elderly), un sito web sul quale l'operatore sanitario potrà gestire i propri pazienti, assegnare esercizi fisici e farmaci, monitorare lo stato di salute e l'umore attraverso una dashboard dedicata.

Oltre alla parte dedicata all'operatore, è stata realizzata un'**interfaccia semplificata per il paziente anziano**: elementi grafici grandi, linguaggio familiare e flussi di interazione lineari. L'anziano può registrare il proprio umore quotidiano ("Come Mi Sento") tramite emoji, eseguire gli esercizi fisici assegnati dall'operatore con istruzioni passo-passo e confermare l'assunzione dei farmaci.

Il nostro obiettivo è quello di offrire un'interfaccia moderna e accessibile, senza trascurarne l'aspetto funzionale: il punto di forza dell'applicazione sarà la doppia vista, una per l'operatore sanitario e una per il paziente, ciascuna ottimizzata per il proprio destinatario.

Le funzionalità principali del sistema comprendono:

- **Gestione Utente:** Gestione e monitoraggio dei profili dei pazienti, gestione del contatto di emergenza e aggiornamento dello stato clinico.
- **Esercizi Fisici:** Creazione e assegnazione di esercizi motori personalizzati con istruzioni passo-passo e tracciamento del completamento.
- **Umore ("Come Mi Sento"):** Registrazione quotidiana dell'umore tramite un'interfaccia intuitiva basata su tre emoji (triste, normale, felice), con storico visualizzabile dall'operatore.
- **Farmaci:** Gestione dei farmaci prescritti con suddivisione per fascia oraria (mattina, pranzo, sera) e conferma di assunzione da parte del paziente.

Crediamo fortemente che questa piattaforma possa rappresentare un vantaggio rispetto alle soluzioni esistenti, che offrono solo funzionalità frammentate (o solo monitoraggio farmaci, o solo dashboard clinica) senza mai integrarle in un unico sistema pensato per l'utenza anziana.

2 Individuazione degli Stakeholder e dei Goal

2.1 Stakeholder e Goal

L'analisi degli stakeholder ha identificato due attori principali che interagiscono con la piattaforma A.L.I.C.E.

Operatore Sanitario / Caregiver: professionista responsabile dell'assistenza clinica e della pianificazione terapeutica. Può trattarsi di un medico, un infermiere o un caregiver professionale. Utilizza la piattaforma per gestire i pazienti, assegnare attività e monitorare il loro stato di salute.

Paziente Anziano: utente fragile che fruisce dei servizi assistenziali dal proprio domicilio. Rappresenta il vincolo progettuale più stringente in termini di usabilità e accessibilità, data la scarsa familiarità con interfacce digitali complesse.

2.1.1 Goal Operatore Sanitario

Stakeholder	Goal	Subgoal	C	R	U	D
Operatore Sanitario	Gestione pazienti	Visualizza lista pazienti		✓		
		Modifica dati paziente			✓	
		Visualizza dettaglio paziente		✓		
	Gestione esercizi	Crea esercizio con passaggi	✓			
		Visualizza esercizi assegnati		✓		
		Elimina esercizio				✓
	Gestione farmaci	Aggiungi farmaco	✓			
		Visualizza farmaci assegnati		✓		
		Elimina farmaco				✓
	Monitoraggio	Visualizza storico umore		✓		
		Visualizza aderenza terapeutica		✓		

Tabella 1: Goal e operazioni CRUD dell'Operatore Sanitario

2.1.2 Goal Paziente Anziano

Stakeholder	Goal	Subgoal	C	R	U	D
Paziente Anziano	Accesso	Login con email e password		✓		
		Registrazione nuovo account	✓			
	Umore	Registra umore quotidiano (emoji)	✓		✓	
		Visualizza selezione corrente		✓		
	Esercizi fisici	Esegui esercizi con passaggi guidati		✓	✓	
	Farmaci	Conferma assunzione farmaco			✓	

Tabella 2: Goal e operazioni CRUD del Paziente Anziano

2.2 Diagramma di Contesto

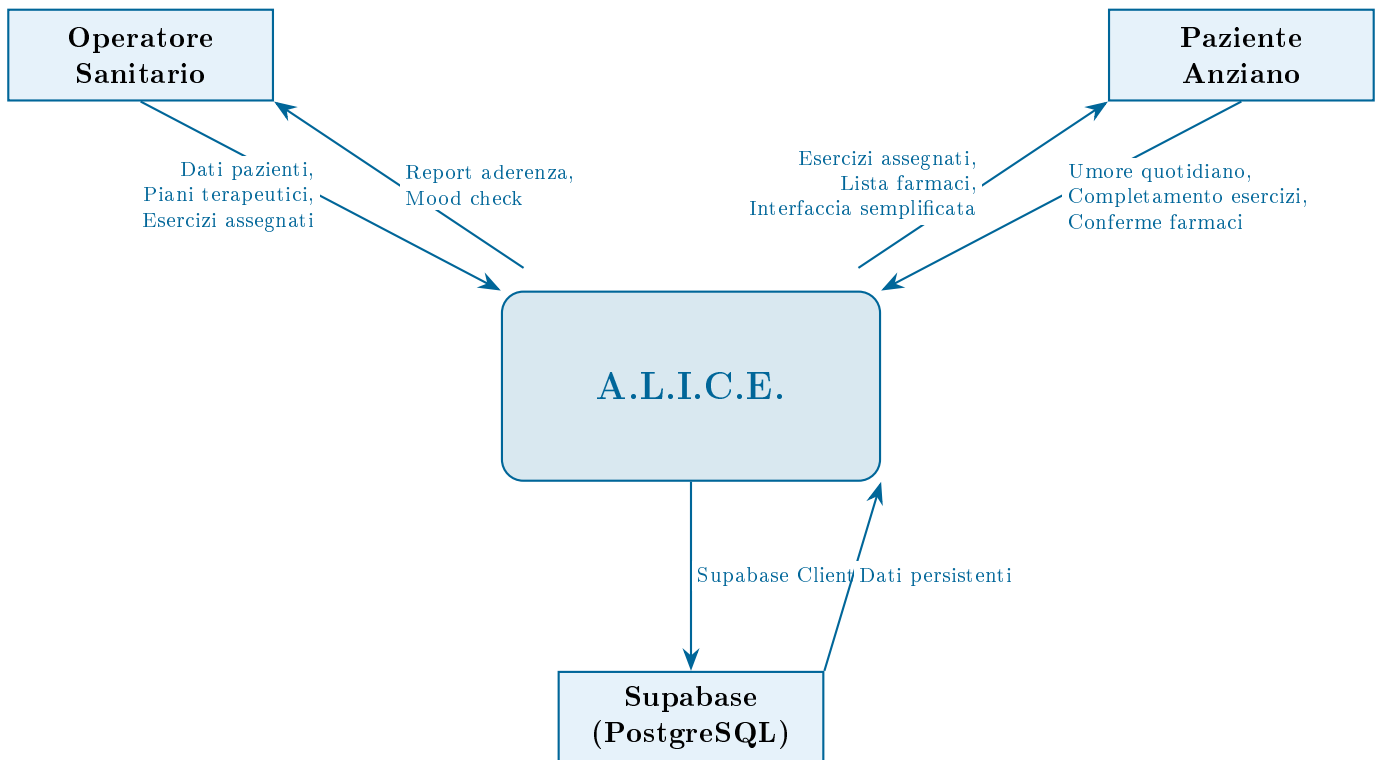


Figura 1: Diagramma di contesto del sistema A.L.I.C.E.

2.3 Scambio di Valore

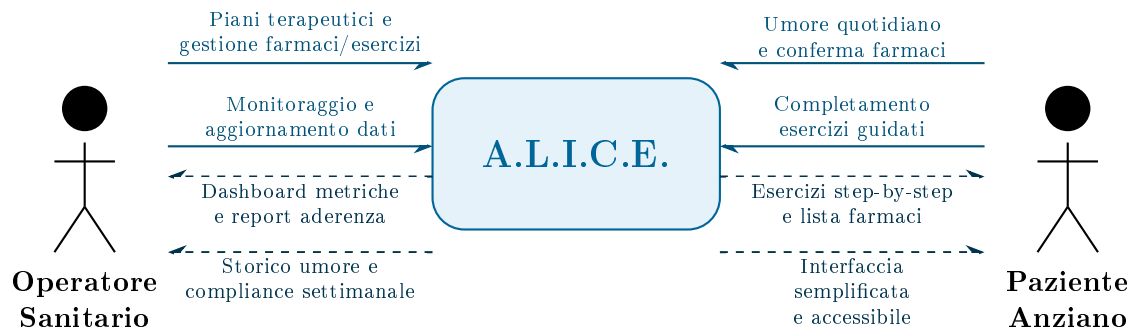


Figura 2: Scambio di valore tra gli stakeholder e il sistema A.L.I.C.E.

3 Stato dell'Arte, Analisi della Concorrenza e Analisi “Make or Buy”

3.1 Stato dell'Arte

Il settore dell'assistenza domiciliare per anziani dispone oggi di diverse soluzioni tecnologiche, ma ciascuna si concentra su un singolo aspetto: alcune puntano sul monitoraggio farmacologico, altre ancora sulla telemedicina. Nessuna offre una visione integrata come quella proposta da A.L.I.C.E.

3.2 Analisi della Concorrenza

Abbiamo analizzato due competitor principali presenti sul mercato, ognuno valido nel proprio ambito ma con limitazioni significative rispetto alla nostra proposta.

3.2.1 MyTherapy

MyTherapy è un'applicazione focalizzata sull'aderenza terapeutica. Gestisce promemoria per i farmaci, consente il tracciamento dei parametri vitali e genera reportistica per il personale medico. È molto precisa nella gestione farmacologica, ma l'esperienza utente ha un taglio “clinico” che può generare ansia nel paziente anziano. Non prevede alcuna forma di stimolazione attiva delle capacità cognitive o fisiche, né dispone di strumenti per il monitoraggio emotivo.

3.2.2 ADiLife

ADiLife è un sistema professionale di teleassistenza per cliniche e strutture sanitarie. Dispone di hardware dedicato per la rilevazione dei parametri e offre un'infrastruttura robusta e sicura. I limiti principali sono l'interfaccia complessa e poco “calda”, inadatta all'uso domestico da parte di anziani, e i costi elevati legati all'hardware e alle licenze professionali. Il focus è sull'emergenza e sulla patologia cronica grave, senza componenti di prevenzione.

Sintesi comparativa

A.L.I.C.E. è l'unica soluzione in grado di coprire simultaneamente tutte le dimensioni rilevanti:

Piattaforma	Attività Fisica	Mood Check	Dashboard OS	Gestione Sanitaria
MyTherapy	×	×	Parziale	✓
ADiLife	×	×	✓	✓
A.L.I.C.E.	✓	✓	✓	✓

Tabella 3: Confronto delle funzionalità tra A.L.I.C.E. e i principali competitor

3.3 Analisi “Make or Buy”

Abbiamo valutato se fosse più conveniente sviluppare l'applicazione internamente o acquistare una soluzione esistente.

Criterio	Make (Sviluppo Interno)	Buy (Soluzione Commerciale)
Personalizzazione	Completa: interfaccia progettata ad hoc per anziani e operatori sanitari.	Limitata: personalizzazioni parametriche insufficienti per le esigenze di accessibilità richieste.
Costi iniziali	Bassi: tecnologie open source (Next.js, Supabase con piano gratuito).	Elevati: licenze annuali per utente con costi crescenti.
Integrazione	Nativa: fisiche ed emotive in un'unica piattaforma.	Frammentata: necessità di integrare più prodotti diversi.
Manutenzione	A carico del team, con pieno controllo sul codice.	Dipendente dal vendor e dalla sua roadmap commerciale.
Privacy e Dati	Pieno controllo sui dati sanitari sensibili.	Dipendente dal provider, spesso con server all'estero.

Tabella 4: Analisi comparativa Make vs Buy

La scelta finale è ricaduta sull'opzione **Make**, principalmente perché nessuna soluzione commerciale copre simultaneamente tutte le dimensioni che A.L.I.C.E. propone (emotiva, fisica, gestionale). L'uso di tecnologie open source mantiene i costi contenuti e ci garantisce il pieno controllo sui dati sanitari.

4 Modello Informativo

4.1 Diagramma EER

Il diagramma Entity-Enhanced Relationship descrive la struttura concettuale della base di dati. Le entità sono state individuate a partire dall'analisi dei requisiti funzionali e riflettono lo schema effettivamente implementato su Supabase (PostgreSQL).

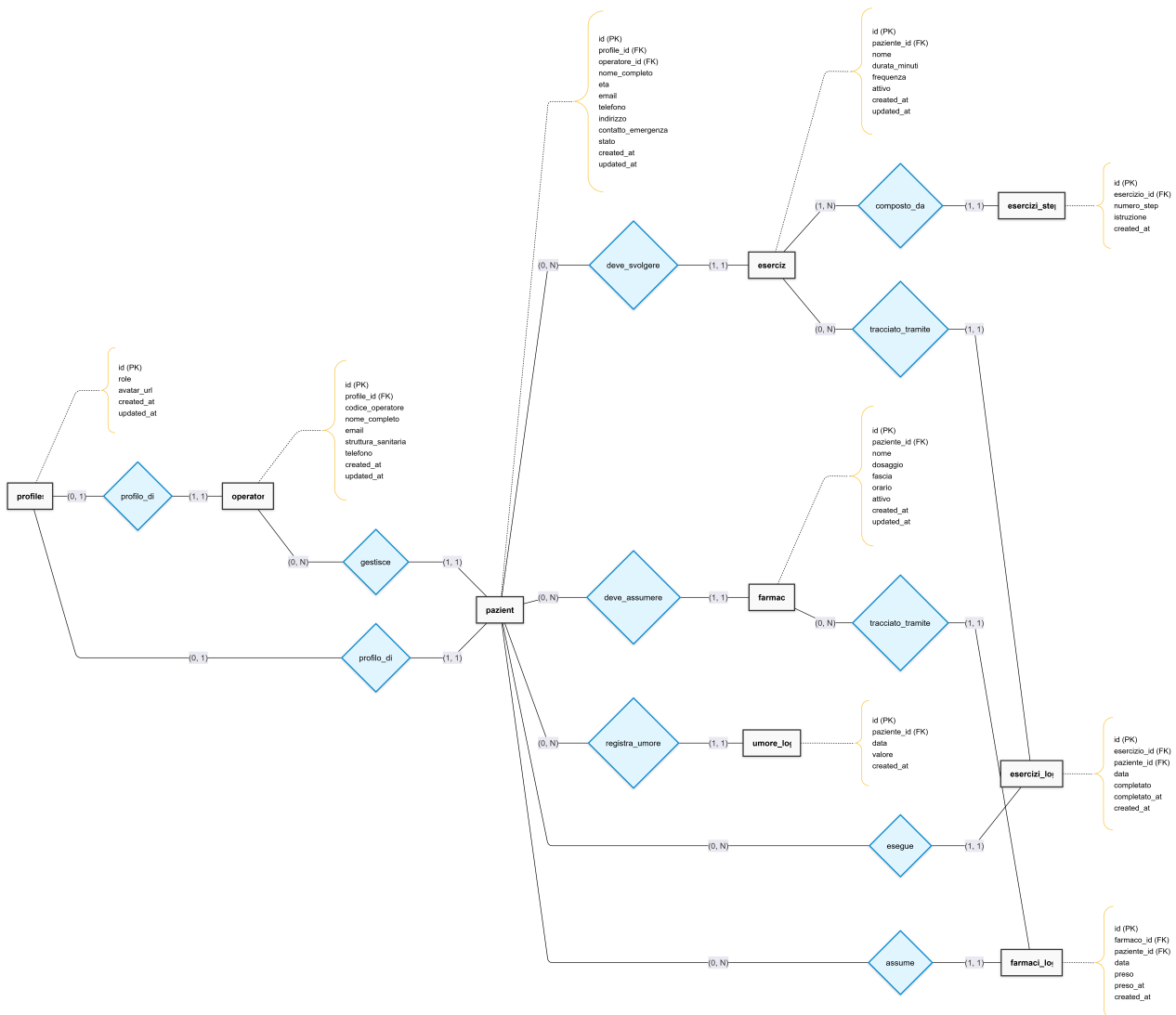


Figura 3: Diagramma EER del sistema A.L.I.C.E.

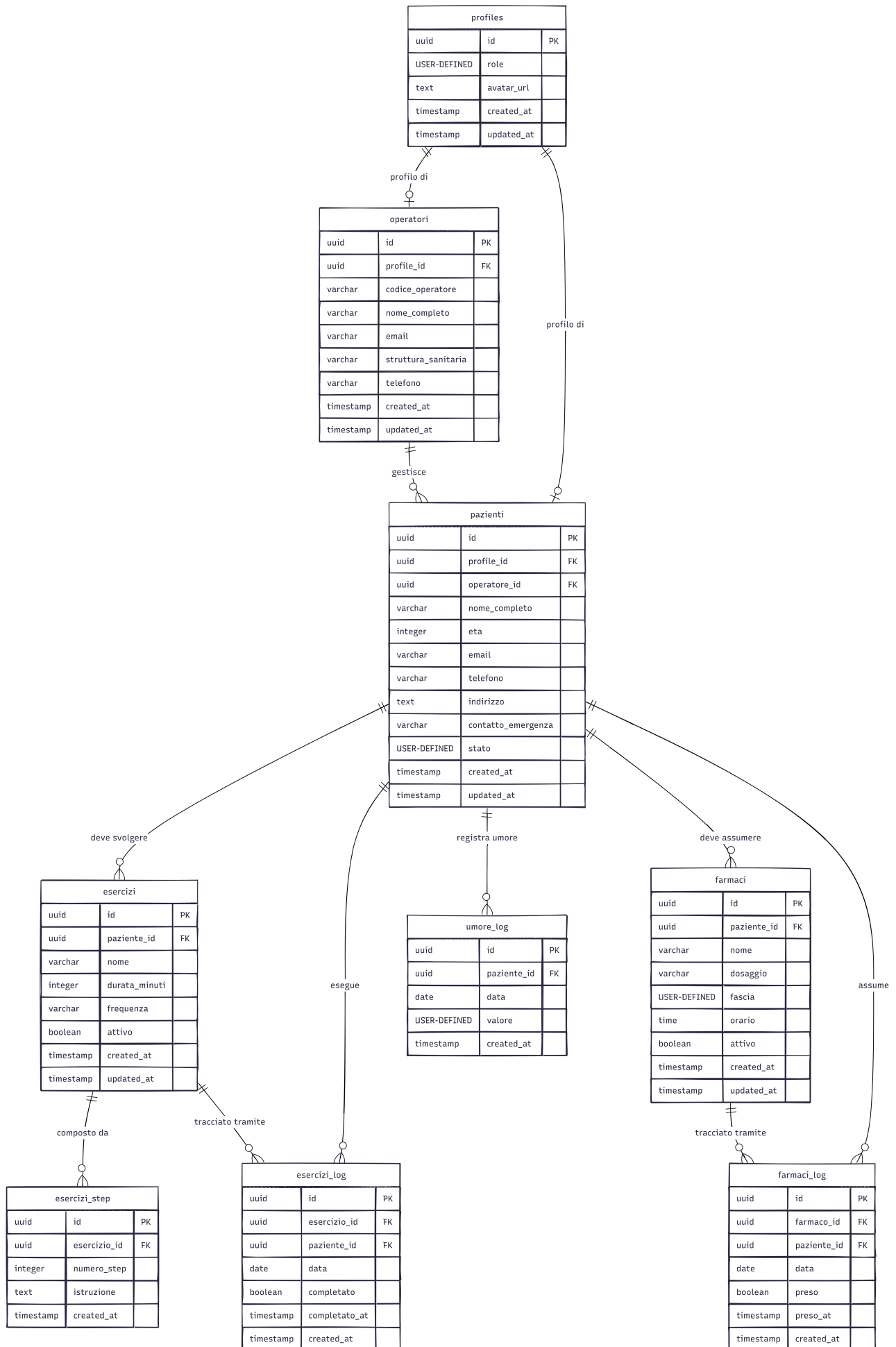
Descrizione delle entità

- **Profiles:** entità generalizzata per autenticazione e ruoli. Attributi: id (UUID, PK, da Supabase Auth), role (“operatore” o “paziente”), avatar_url, created_at, updated_at.
- **Pazienti:** specializzazione di Profiles. Attributi: id, profile_id (FK), operatore_id (FK), nome_completo, eta, email, telefono, indirizzo, contatto_emergenza, stato (“ok”/“attenzione”/“critico”), created_at, updated_at.
- **Operatori:** specializzazione di Profiles. Attributi: id, profile_id (FK), codice_operatore, nome_completo, email, struttura_sanitaria, telefono, created_at, updated_at.

- **Esercizi:** esercizio fisico assegnato al paziente. Attributi: `id`, `paziente_id` (FK), `nome`, `durata_minuti`, `frequenza`, `attivo`, `created_at`, `updated_at`.
- **Esercizi Step:** singolo passaggio di un esercizio. Attributi: `id`, `esercizio_id` (FK), `numero_step`, `istruzione`, `created_at`.
- **Esercizi Log:** registro completamento esercizi. Attributi: `id`, `esercizio_id` (FK), `paziente_id` (FK), `data`, `completato`, `completato_at`, `created_at`.
- **Farmaci:** farmaco prescritto. Attributi: `id`, `paziente_id` (FK), `nome`, `dosaggio`, `fascia` (mattina/pranzo/sera, calcolata automaticamente dall'orario), `orario`, `attivo`, `created_at`, `updated_at`.
- **Farmaci Log:** registro assunzione farmaci. Attributi: `id`, `farmaco_id` (FK), `paziente_id` (FK), `data`, `preso`, `preso_at`, `created_at`.
- **Umore Log:** registrazione quotidiana dell'umore. Attributi: `id`, `paziente_id` (FK), `data`, `valore` ("triste"/"normale"/"felice"), `created_at`.

4.2 Modello Relazionale

Il modello relazionale descrive la struttura logica della base di dati, derivato dal diagramma EER. Le tabelle, le chiavi primarie (PK), le chiavi esterne (FK) e i vincoli di integrità referenziale sono rappresentati nel seguente schema:



4.3 Stima Dimensionale del Database

4.3.1 Stima per entità

Tabella	Byte/riga (stimati)	Note
profiles	128	UUID + ruolo + avatar_url
pazienti	512	Dati anagrafici + stato clinico + contatto emergenza
operatori	384	Dati professionali + codice operatore
esercizi	192	Nome + parametri + FK
esercizi_step	320	Numero step + testo istruzione
esercizi_log	96	FK esercizio + data + booleano completamento
farmaci	208	Nome + dosaggio + fascia + orario
farmaci_log	96	FK farmaco + data + booleano preso + timestamp
umore_log	80	Data + valore (enum testuale)

Tabella 5: Stima della dimensione media per riga di ciascuna tabella

4.3.2 Tabella di popolamento e crescita

Tabella	N. righe (1° mese)	Byte/riga	Dim. iniziale	Crescita annua
profiles	105	128	~ 13 KB	trascurabile
pazienti	100	512	~ 50 KB	trascurabile
operatori	5	384	~ 2 KB	trascurabile
esercizi	300	192	~ 56 KB	~ 338 KB
esercizi_step	900	320	~ 281 KB	~ 1,6 MB
esercizi_log	3.000	96	~ 281 KB	~ 3,3 MB
farmaci	500	208	~ 102 KB	~ 305 KB
farmaci_log	4.500	96	~ 422 KB	~ 5,1 MB
umore_log	3.000	80	~ 234 KB	~ 2,7 MB
Totale stimato (primo mese)			~ 1,4 MB	~ 13,3 MB/anno

Tabella 6: Stima del popolamento iniziale e crescita annuale del database

La dimensione è gestibile con il piano base di Supabase (500 MB inclusi nel piano gratuito).

4.3.3 Modello di visibilità

Il modello di visibilità definisce le policy di accesso ai dati in base al ruolo dell'utente autenticato. Il sistema implementa un modello **RBAC** (Role-Based Access Control) a due livelli: ruolo (operatore vs paziente) e ownership (ogni utente accede solo ai dati di propria competenza).

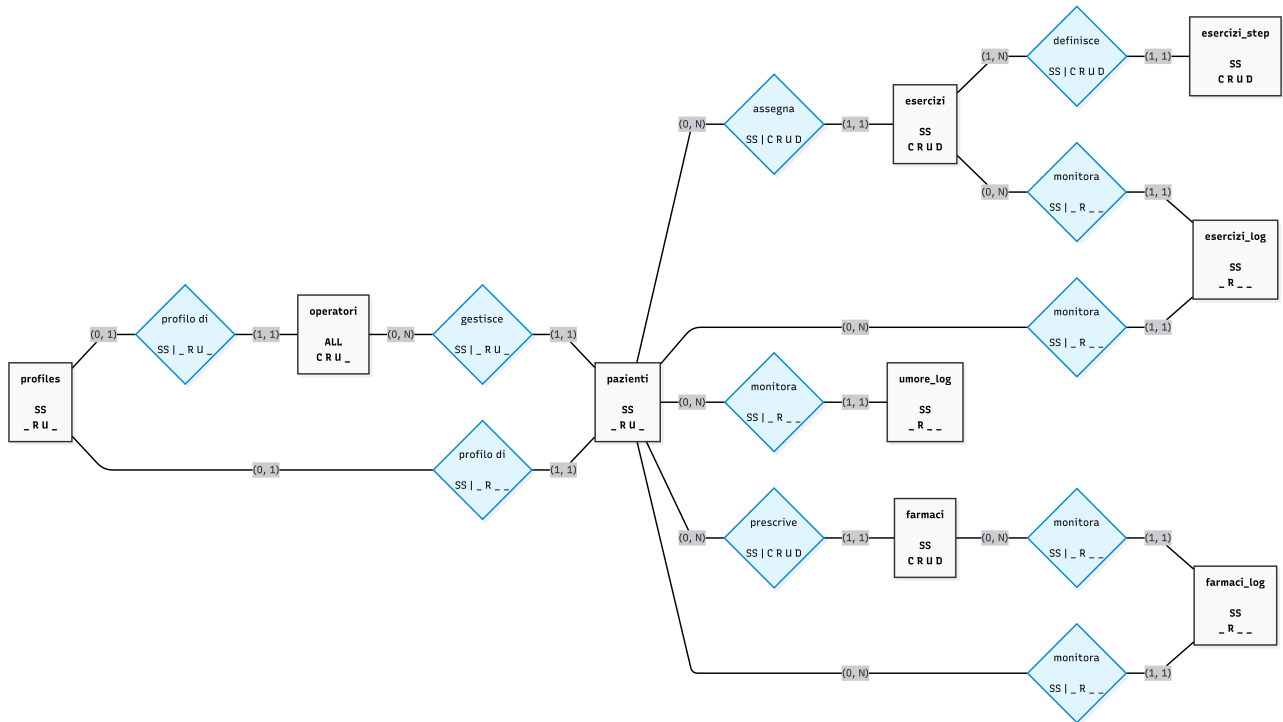


Figura 5: Modello di visibilità per l'Operatore Sanitario

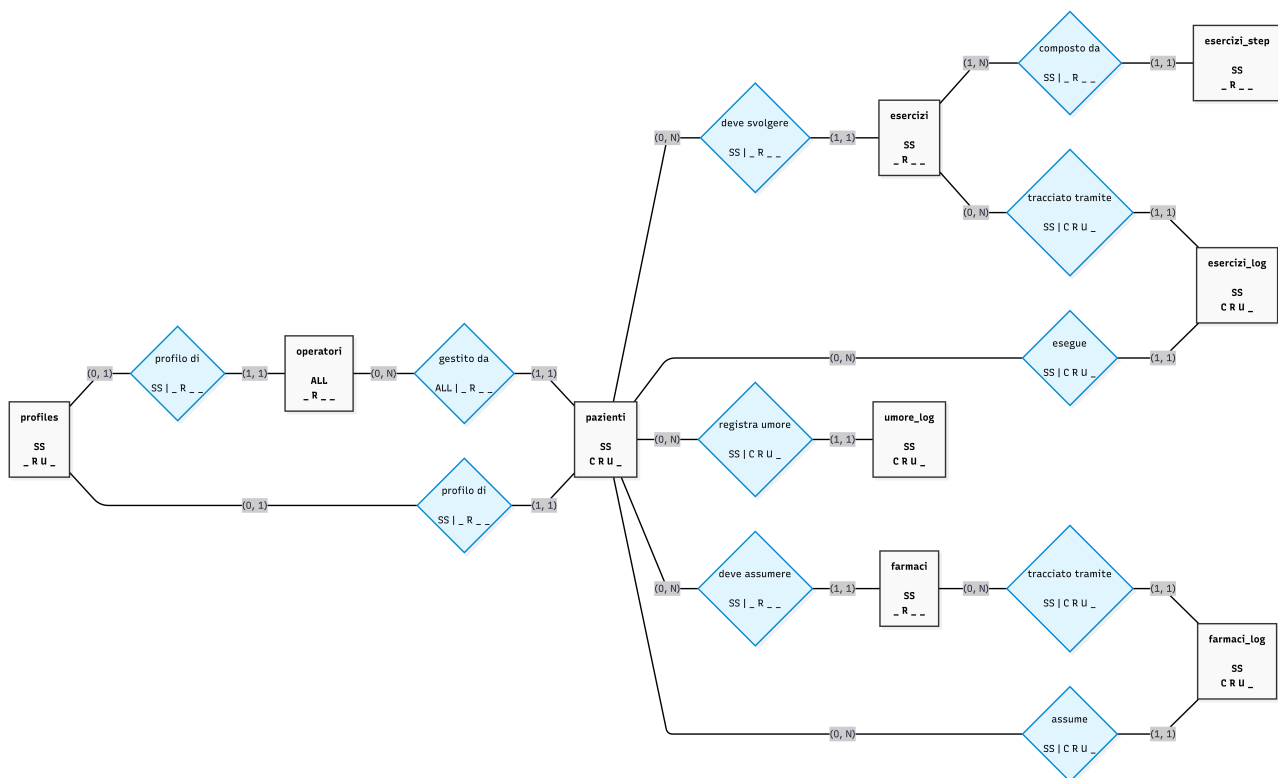


Figura 6: Modello di visibilità per il Paziente Anziano

5 Modello di Navigazione

5.1 Albero di Navigazione

Il sistema A.L.I.C.E. prevede due aree completamente separate: una per l'**operatore sanitario** e una per il **paziente anziano**. L'accesso avviene dalla landing page principale, dalla quale l'utente sceglie il proprio ruolo e viene indirizzato alla pagina di autenticazione appropriata. La struttura complessiva dell'applicazione è rappresentata nel seguente albero di navigazione:

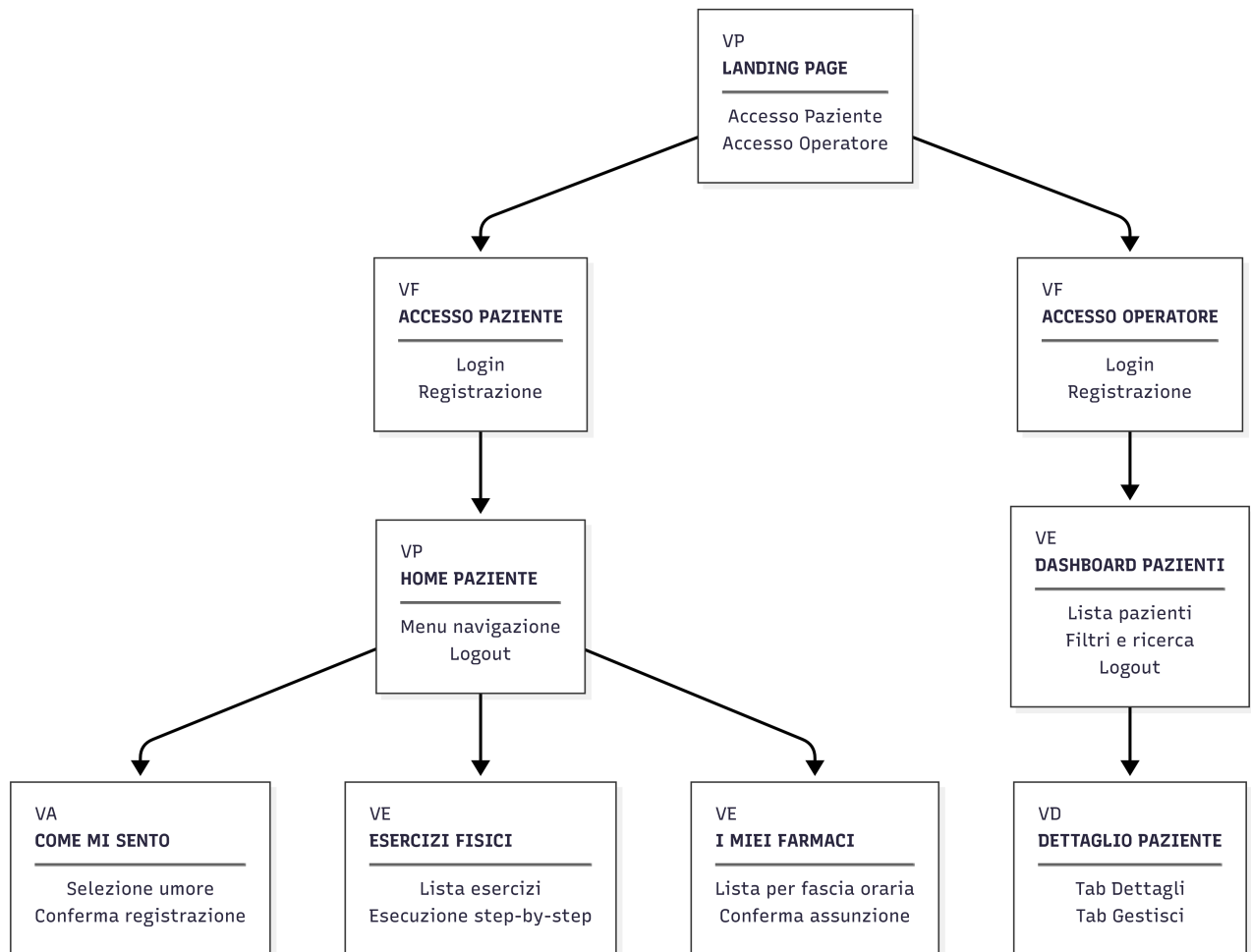


Figura 7: Albero di navigazione del sistema A.L.I.C.E.

5.2 Legenda Tipi di Vista

Sigla	Tipo	Descrizione
VP	Vista Pagina	Pagina informativa o di selezione
VF	Vista Form	Pagina con form di inserimento dati
VE	Vista Elenco	Pagina con lista di elementi
VD	Vista Dettaglio	Pagina con informazioni dettagliate su un singolo elemento
VA	Vista Azione	Pagina con interazione diretta (selezione, conferma)

Tabella 7: Legenda dei tipi di vista utilizzati nell'albero di navigazione

5.3 Descrizione delle Viste

5.3.1 Area comune

- **Landing Page** (*VP*): pagina iniziale del sistema con scelta del ruolo. Presenta due pulsanti per accedere all'area operatore o all'area paziente.

5.3.2 Area Operatore Sanitario

- **Accesso Operatore** (*VF*): pagina con due tab (Login e Registrazione). Il login richiede email e password; la registrazione richiede nome completo, email, password, struttura sanitaria e telefono.
- **Dashboard Pazienti** (*VE*): vista elenco con la lista dei pazienti associati all'operatore. Presenta contatori per stato clinico (critico/attenzione/ok), barra di ricerca per nome e filtri per stato. Ogni paziente è rappresentato da una card con metriche sintetiche (aderenza farmaci, compliance esercizi).
- **Dettaglio Paziente** (*VD*): vista dettaglio del singolo paziente, organizzata in due tab:
 - *Tab Dettagli*: metriche di aderenza farmaci e compliance esercizi, storico umore settimanale, storico farmaci e storico esercizi degli ultimi 7 giorni.
 - *Tab Gestisci*: form di modifica dei dati anagrafici del paziente, gestione farmaci (creazione ed eliminazione con nome, dosaggio e orario – la fascia oraria viene calcolata automaticamente), gestione esercizi (creazione ed eliminazione con nome, durata, frequenza e passaggi guidati step-by-step).

5.3.3 Area Paziente Anziano

- **Accesso Paziente** (*VF*): pagina con due tab (Login e Registrazione). Il login richiede email e password; la registrazione richiede la selezione dell'operatore di riferimento, nome completo, email, password, età, telefono, indirizzo e contatto di emergenza.
- **Home Paziente** (*VP*): pagina hub con tre card grandi e colorate che permettono di navigare verso le sezioni principali (Umore, Esercizi, Farmaci). Presente pulsante di logout.

- **Come Mi Sento** (*VA*): vista per la registrazione dell'umore quotidiano. Presenta tre opzioni di selezione (triste, normale, felice) con possibilità di aggiornare la scelta durante la giornata. Al termine mostra una schermata di conferma.
- **Esercizi Fisici** (*VE + VD*): lista degli esercizi assegnati con nome e durata. Al click su un esercizio, si accede all'esecuzione guidata passo-passo con istruzioni dettagliate e pulsante di completamento.
- **I Miei Farmaci** (*VE*): elenco dei farmaci prescritti, organizzato per fascia oraria (mattina, pranzo, sera). Ogni farmaco mostra nome, dosaggio e orario, con pulsante di conferma assunzione. Un contatore indica i farmaci già presi nella giornata.

5.4 Principi di Navigazione

- Massimo **due livelli di profondità** dalla pagina principale di ciascuna area.
- Pulsante **“Torna indietro”** sempre visibile in ogni sotto-pagina.
- **Nessun menu hamburger**: tutta la navigazione avviene tramite pulsanti e card visibili.
- **Feedback visivo immediato** per ogni azione dell'utente (conferma, errore, caricamento).
- Interfaccia paziente ottimizzata per **accessibilità**: testi grandi, pulsanti ampi, icone intuitive.

6 Modello Architettuale

6.1 Architettura Hardware

Il sistema A.L.I.C.E. è progettato per operare su un'infrastruttura **serverless/managed**, sfruttando Supabase come Backend-as-a-Service e Vercel per il deploy del frontend Next.js. Non è necessario gestire server fisici.

Componente	Servizio	Dettaglio
Frontend / SSR	Vercel	Deploy automatico da repository Git
Database (PostgreSQL)	Supabase	Istanza PostgreSQL managed
Autenticazione	Supabase Auth	Email e password (entrambi i ruoli)
API REST	Supabase (PostgREST)	API auto-generate dalle tabelle

Tabella 8: Infrastruttura cloud del sistema A.L.I.C.E.

I dispositivi client accedono tramite browser web. Il paziente anziano dovrebbe disporre di un tablet da almeno 10" o un PC desktop; l'operatore sanitario può usare PC, laptop o tablet. Non è richiesta l'installazione di software aggiuntivo.

6.2 Architettura Software

L'architettura usa Next.js come framework full-stack e Supabase come backend gestito.

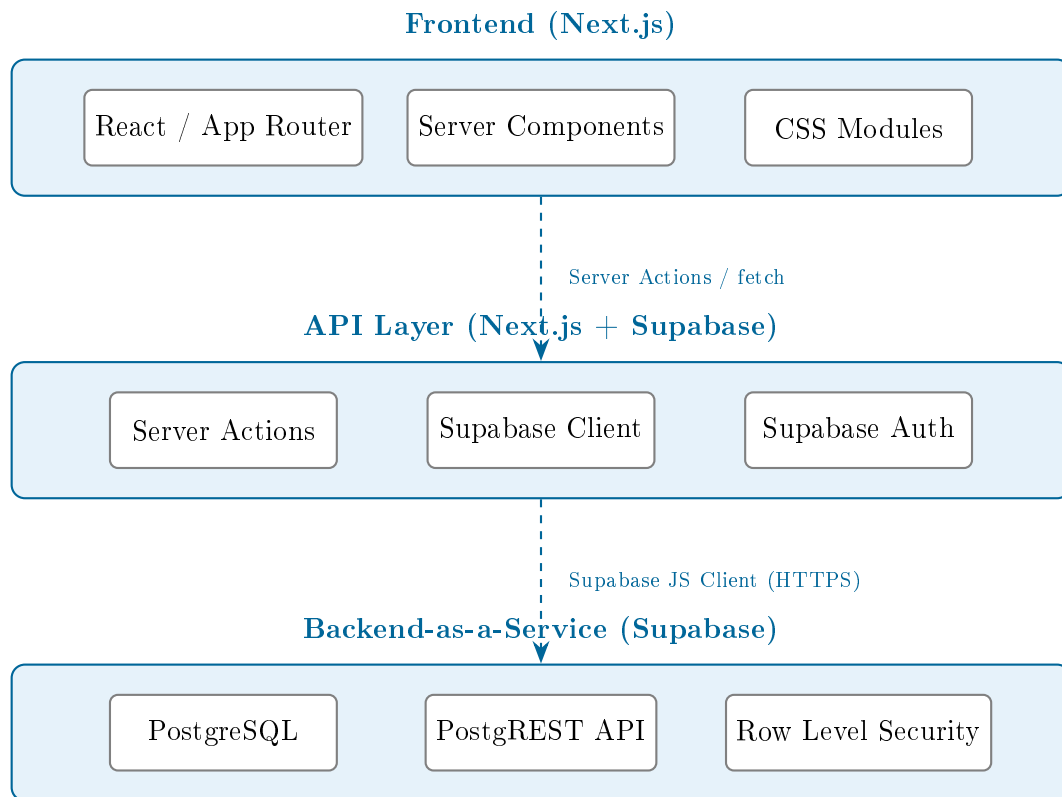


Figura 8: Architettura software del sistema A.L.I.C.E.

6.2.1 Stack Tecnologico

- **Next.js 16** → Framework React full-stack. App Router con routing basato su file system, Server Components per ridurre il JavaScript lato client, Server Actions per le mutazioni dati.
- **React 19 / React-DOM 19** → Libreria UI per la costruzione dei componenti.
- **Supabase** → Backend-as-a-Service open source basato su PostgreSQL. Fornisce database managed, API auto-generate (PostgREST), autenticazione, Row Level Security e storage file.
- **@supabase/supabase-js** → Client JavaScript per l'interazione con Supabase.
- **@supabase/ssr** → Integrazione Supabase con il rendering server-side di Next.js, gestione cookie di sessione.
- **CSS Modules** → Styling con scope locale per componente, nessuna dipendenza esterna per il CSS.

6.2.2 DBMS

Il database è **PostgreSQL 15**, gestito da Supabase. L'accesso avviene tramite le API auto-generate (PostgREST) per le operazioni CRUD standard, oppure tramite SQL diretto per

operazioni complesse, stored procedures e policy RLS. PostgreSQL è stato scelto per la sua maturità, l'assenza di vendor lock-in e il supporto nativo a Row Level Security.

6.2.3 Browser e Sistemi Operativi Supportati

Essendo un'applicazione web, il sistema funziona su qualsiasi sistema operativo con un browser moderno. I browser supportati sono: Chrome 90+, Firefox 88+, Edge 90+, Safari 14+, Samsung Internet 14+. Il design responsive si adatta a schermi da 768×1024 pixel (tablet) fino a risoluzioni Full HD e superiori.

7 Aspetti Implementativi

7.1 Struttura del Progetto

Il progetto è organizzato secondo la struttura standard di un'applicazione Next.js con App Router, con una directory `components/` dedicata ai componenti riutilizzabili. Di seguito i file e le cartelle principali:

7.1.1 Pagine (app/)

- `app/layout.tsx` → Layout radice dell'applicazione, include i font e i metadata condivisi.
- `app/page.tsx` → Landing page con selezione del ruolo (Operatore / Paziente).
- `app/operatore/login/page.tsx` → Login operatore con email e password.
- `app/operatore/registratori/page.tsx` → Registrazione operatore con dati professionali.
- `app/operatore/dashboard/layout.tsx` → Layout con header per la vista operatore (nome, codice, logout).
- `app/operatore/dashboard/page.tsx` → Dashboard con lista pazienti, filtri, ricerca e card metriche.
- `app/operatore/dashboard/loading.tsx` → Schermata di caricamento per feedback visivo istantaneo.
- `app/operatore/dashboard/pazienti/[id]/page.tsx` → Dettaglio paziente con tab (dettagli/gestisci).
- `app/operatore/dashboard/pazienti/[id]/loading.tsx` → Schermata di caricamento per il dettaglio paziente.
- `app/paziente/login/page.tsx` → Login paziente con email e password.
- `app/paziente/registratori/page.tsx` → Registrazione paziente con selezione operatore.
- `app/paziente/home/page.tsx` → Home con card colorate per navigazione (umore, esercizi, farmaci).
- `app/paziente/umore/page.tsx` → Registrazione umore quotidiano con selezione tra tre opzioni (triste, normale, felice).
- `app/paziente/esercizi/page.tsx` → Lista esercizi assegnati con esecuzione guidata step-by-step.
- `app/paziente/farmaci/page.tsx` → Elenco farmaci organizzato per fascia oraria con conferma assunzione.

7.1.2 Componenti Riutilizzabili (components/)

- `components/DashboardLayoutClient.tsx` → Client component per l'header della dashboard operatore con logout.
- `components/DashboardContent.tsx` → Client component con filtri, ricerca e griglia di card pazienti.
- `components/PatientCard.tsx` → Card riepilogativa di un paziente con metriche sintetiche.
- `components/PazienteDettaglio.tsx` → Vista dettaglio paziente con tab, form dati anagrafici, CRUD farmaci ed esercizi.

7.1.3 Lato Server (lib/ e actions)

- `lib/supabase/client.ts` → Configurazione del client Supabase lato browser con gestione dei cookie di sessione.
- `lib/supabase/server.ts` → Configurazione del client Supabase lato server con accesso ai cookie tramite Next.js.
- `app/operatore/actions.ts` → Server Actions per login, registrazione e logout operatore.
- `app/operatore/dashboard/pazienti/actions.ts` → Server Actions per la gestione dei pazienti: aggiornamento dati anagrafici, CRUD farmaci (con calcolo automatico della fascia oraria dall'orario di assunzione), CRUD esercizi con passaggi guidati.
- `app/paziente/actions.ts` → Server Actions per login, registrazione e logout paziente.
- `middleware.ts` → Middleware Next.js per il refresh automatico della sessione Supabase, protezione delle rotte autenticate e redirect basato sul ruolo (operatore → dashboard, paziente → home).

7.2 Sicurezza

7.2.1 Autenticazione

Il sistema utilizza **Supabase Auth** per la gestione delle sessioni:

- Sia l'operatore che il paziente accedono tramite **email e password**.
- Alla registrazione, viene creato un profilo in **auth.users** con un trigger PostgreSQL che popola automaticamente la tabella **profiles** con il ruolo selezionato.
- Le sessioni sono gestite tramite JWT con refresh automatico. Il middleware Next.js aggiorna il cookie di sessione ad ogni richiesta.
- Le password sono hashate con **bcrypt** dal servizio Supabase Auth.

7.2.2 Row Level Security (RLS)

La sicurezza a livello di dato è implementata con le policy RLS di PostgreSQL. Ogni tabella ha regole che garantiscono che un paziente possa vedere solo i propri dati e un operatore possa accedere solo ai dati dei pazienti a lui assegnati.

8 Test

Per il test del sistema A.L.I.C.E. sono stati utilizzati i seguenti strumenti:

- **Postman** – Applicazione utilizzata per effettuare test funzionali sulle API di Supabase, verificandone il corretto funzionamento simulando richieste HTTP (GET, POST, PATCH, DELETE) verso gli endpoint REST auto-generati da PostgREST.
- **ESLint** – Strumento di analisi statica del codice JavaScript/TypeScript, utilizzato per individuare e correggere bug, errori di sintassi e problemi di stile, migliorando la qualità e la coerenza del codice sorgente.
- **Supabase Dashboard (SQL Editor)** – Interfaccia web fornita da Supabase per eseguire query SQL dirette sul database PostgreSQL, utilizzata per verificare la corretta persistenza dei dati e il funzionamento delle policy Row Level Security.
- **Chrome DevTools** – Strumenti di sviluppo integrati nel browser Google Chrome, utilizzati per il debug del frontend React, l'analisi delle richieste di rete, il controllo del rendering dei componenti e la verifica della responsività dell'interfaccia.

9 Conclusioni

Il progetto A.L.I.C.E. nasce per rispondere a un'esigenza concreta: offrire uno strumento digitale integrato per l'assistenza domiciliare agli anziani, che metta in comunicazione il paziente e l'operatore sanitario attraverso un'unica piattaforma.

Il risultato è un'applicazione web composta da 4 componenti React riutilizzabili, 11 pagine distribuite tra le due aree (operatore e paziente), 3 file di Server Actions per le operazioni sui dati, e un database PostgreSQL con 9 tabelle. L'intera piattaforma si appoggia su tecnologie open source (Next.js 16, React 19, Supabase) con costi di infrastruttura praticamente nulli nella fase iniziale grazie ai piani gratuiti.

I punti di forza principali sono:

- La **doppia interfaccia**: una dashboard analitica per l'operatore con metriche di aderenza farmaci e compliance esercizi, e un'interfaccia semplificata per il paziente anziano con elementi grafici grandi e pulsanti intuitivi.
- L'**integrazione** di funzionalità normalmente separate: monitoraggio dell'umore, esercizi fisici guidati passo-passo e gestione farmaci con organizzazione per fascia oraria, il tutto in un unico sistema coerente.
- L'**accessibilità**: elementi grafici grandi, flussi lineari con massimo due livelli di profondità e linguaggio familiare per l'utenza anziana. Ogni sezione del paziente è raggiungibile in massimo due tocchi dalla home.

9.1 Sviluppi futuri

Tra gli sviluppi futuri si possono considerare:

- Integrazione con dispositivi IoT wearable per il monitoraggio automatico dei parametri vitali.
- Ampliamento della libreria di esercizi con contenuti multimediali (video dimostrativi).
- Sistema di notifiche push per promemoria automatici su farmaci e attività.
- Versione app mobile nativa per una maggiore integrazione con il dispositivo.
- Grafici interattivi per l'andamento nel tempo delle metriche nella dashboard operatore.

10 Matrice RACI

La matrice RACI (Responsible, Accountable, Consulted, Informed) definisce le responsabilità rispetto alle principali attività del progetto. Il team è composto da tre membri.

10.1 Matrice – Documentazione

Attività	Manna	Mininni	Tamuliunaite
Descrizione del contesto e degli obiettivi	R/A	C	I
Individuazione stakeholder e goal	I	R/A	C
Diagramma di contesto	C	I	R/A
Stato dell'arte e analisi concorrenza	R/A	C	I
Analisi "Make or Buy"	I	R/A	C
Modello informativo (EER, relazionale, stima DB)	R/A	R/A	R/A
Modello di visibilità (RBAC)	R/A	C	I
Modello di navigazione	I	R/A	C
Modello architetturale (HW e SW)	C	I	R/A
Aspetti implementativi	R/A	C	I
Sezione test e strumenti di verifica	I	R/A	C
Conclusioni e sviluppi futuri	C	I	R/A
Matrice RACI	R/A	R/A	R/A
Revisione finale della documentazione	R/A	R/A	R/A

Tabella 9: Matrice RACI – Documentazione

10.2 Matrice – Sviluppo

Attività	Manna	Mininni	Tamuliunaite
Landing page (selezione ruolo)	I	I	R/A
Login e registrazione operatore	R/A	I	I
Login e registrazione paziente	I	I	R/A
Dashboard operatore (lista pazienti, filtri, ricerca)	R/A	C	I
Dettaglio paziente (tab dettagli e gestisci)	C	R/A	I
Gestione farmaci (CRUD con fascia oraria automatica)	I	R/A	C
Gestione esercizi (CRUD con step guidati)	I	R/A	C
Registrazione umore quotidiano (“Come Mi Sento”)	I	I	R/A
Interfaccia farmaci paziente (conferma assunzione)	I	C	R/A
Interfaccia esercizi paziente (esecuzione step-by-step)	I	C	R/A
Componenti riutilizzabili (PatientCard, DashboardContent, ecc.)	R/A	C	I
Server Actions operatore (auth, CRUD pazienti)	R/A	R/A	I
Server Actions paziente (auth, umore, esercizi, farmaci)	I	I	R/A
Middleware Next.js (sessioni, redirect, protezione rotte)	R/A	I	C
Styling CSS Modules e design responsive	C	I	R/A

Tabella 10: Matrice RACI – Sviluppo

Legenda: **R** = Responsible (esegue), **A** = Accountable (approva), **C** = Consulted (fornisce input), **I** = Informed (informato).